Chapter 13

Markdown Writing Instructions

Panaversity — General Agents Foundations

MCQ Exam Preparation Notes

Complete Study Notes

Prepared for: Imran Hassan | 2026

Chapter Overview

Yeh chapter sikhata hai ke Markdown ko AI agents ke liye instructions likhne mein kaise use kiya jata hai. Markdown sirf formatting nahi — yeh **Intent Layer** hai AI-Driven Development mein.

> **Core Message:** "Markdown ek specification language hai jo aap AI agents ko batane ke liye use karte ho ke kya build karna hai."

Aap (Intent Layer) → Markdown Spec (Structure) → AI Reads (Reasoning) → Better Code (Implementation)

5 Lessons

1. Introduction — Why Markdown Matters
2. Headings — Document Hierarchy
3. Lists — Organizing Ideas
4. Code Blocks — Showing Examples
5. Links, Images & Emphasis

Markdown Ki History

Detail	Info
Created	2004, John Gruber ne banaya
CommonMark	2014 mein formal specification — base standard
GitHub Flavored Markdown (GFM)	Tables, task lists, strikethrough add karta hai
File Extension	.md

Lesson 1: Introduction — Why Markdown Matters

Structured vs Unstructured Text

❌ Unstructured (Galat)

"Hey I need an app for tracking tasks. Users should be able to add tasks and see them and delete them. When they open there should be a menu..."

AI guess karta hai — features kya hain? Order kya hai?

✅ Structured (Sahi)

```
# Task Tracker App
Features:
- Add new tasks
- View all tasks
- Delete tasks
```

AI instantly 3 features aur structure samajh gaya!

> **"Same information, dramatically different output. Structure removes ambiguity."**

AIDD ke 3 Layers

Layer	Kaun Kaam Karta Hai	Kya Hota Hai
Layer 1: Intent	**AAP**	Markdown mein spec likho — WHAT aap chahte ho
Layer 2: Reasoning	AI	Code structure, libraries, implementation plan decide kare
Layer 3: Implementation	AI	Actual code likhe jo spec se match kare

> **Important:** Specification = design authority. Implementation must match the spec, not the other way around. Aap Layer 1 ke final authority ho.

Technical Reason — Kyun Kaam Karta Hai?

LLMs text ko "tokens" mein process karte hain. Markdown headings jaise `## Features` act as **attention cues** — model ko bata deta hai ke "yahan se features start hote hain."

- Lists natural separations create karti hain items ke beech
- Better structure = better AI comprehension
- AI ko zyada guess nahi karna padta

Markdown Kahan Likhein (Free Tools)

Tool	Kaise	Best For
StackEdit	stackedit.io — browser mein, install nahi	Beginners
VS Code	Ctrl+Shift+V for live preview	Developers
GitHub	README.md edit karo, Preview tab	Projects

AI Responses Verify Karne Ka 4-Step Framework

1. **Check against what you know** — AI ka jawab manually verify karo
2. **Ask AI to explain reasoning** — "Why is this correct?" poochho
3. **Test specific claims** — Khud render karke dekho
4. **Cross-reference** — Doosre AI tool se confirm karo

Lesson 2: Headings — Document Hierarchy

Hash Symbols — Heading Levels

```
# Level 1 Heading   → Sabse bada — SIRF EK baar
## Level 2 Heading  → Main sections
### Level 3 Heading → Subsections
#### Level 4 Heading → Deep details
##### Level 5       → Avoid karo
###### Level 6      → Avoid karo
```

Har Level Ka Kaam

Level	Symbol	Purpose
1	`#`	Document title — SIRF EK baar
2	`##`	Main sections (Problem, Features, Installation)
3	`###`	Subsections within a section
4	`####`	Deep technical details, constraints

5-6	#####	Avoid — document too complex hai

Sahi Hierarchy Example

```
# Task Tracker App          ← Level 1 (Title)
## Problem                  ← Level 2 (Section)
## Features                 ← Level 2 (Section)
### Add Tasks               ← Level 3 (Subsection)
### View Tasks              ← Level 3 (Subsection)
## Installation             ← Level 2 (Section)
```

3 Common Mistakes

❌ Mistake 1: Space Bhool Gaye

```
#Heading Without Space
```

✅ Sahi

```
# Heading With Space
```

❌ Mistake 2: Multiple H1

```
# My App
# Problem
# Features
```

✅ Sahi

```
# My App
## Problem
## Features
```

❌ Mistake 3: Level Skip

```
# My App
### Installation  ← Level 2 skip!
```

✅ Sahi

```
# My App
## Installation
### Steps
```

> **Kyun Important Hai:** Level skip karne se AI ka logical map toot jaata hai. Screen readers bhi fail ho jaate hain — accessibility break hoti hai.

AI ke Liye Headings Ka Faida

- Document ki 4 main sections instantly parse karta hai
- Features section mein jaata hai seedha

- Completeness validate karta hai — "Problem section hai?"
- Better code generate karta hai — "Features list se functions pata chalte hain"

Lesson 3: Lists — Ideas Organize Karna

Unordered List (Bullet Points)

Jab order matter nahi karta — features, requirements, options:

```
- First item     (dash)
* First item     (asterisk)
+ First item     (plus)

Nested list (2 spaces indent):
- Main feature
  - Sub-feature A
  - Sub-feature B
```

Ordered List (Numbers)

Jab sequence zaroori ho — installation, steps, workflow:

```
1. Install Python 3.9
2. Download project files
3. Install packages: pip install requests
4. Run: python app.py
```

> **Warning:** Markdown first number se auto-number karta hai. Agar `11.` se start karo to 12, 13... render hoga. AI agents raw file padhte hain — hamesha correct sequential numbers likho.

Kab Kaunsa Use Karein

Unordered (`-`) Use Karo	Ordered (`1.`) Use Karo
App features describe karna	Installation steps

Requirements ya constraints	Workflow ya process
Options ya alternatives	Troubleshooting steps
Order koi bhi ho — same result	Sequence affect kare outcome ko

> **Decision Rule:** "Does order matter?" — Haan = Ordered, Nahi = Unordered

3 Common List Mistakes

❌ **Space Bhool Gaye**

```
-No space
1.No space
```

✅ **Sahi**

```
- Space hai
1. Space hai
```

❌ **Features ke liye Ordered**

```
## Features
1. Dark mode
2. Search
```

✅ **Sahi**

```
## Features
- Dark mode
- Search
```

❌ **Steps ke liye Unordered**

```
## Installation
- Install Python
- Run program
```

✅ **Sahi**

```
## Installation
1. Install Python
2. Run program
```

AI ke Liye Lists Ka Faida

- Unordered under "Features" = parallel capabilities — modular architecture
- Ordered under "Installation" = sequential scripts generate karta hai
- Ordered lists = dependencies communicate karte hain (Python pehle, phir packages)

Nested Lists vs More Headings

> **Rule:** Major sections ke liye headings. Section ke andar details ke liye nested lists. Agar 10+ Level 3 headings bana rahe ho — nested lists use karo!

Lesson 4: Code Blocks — Examples Dikhana

Problem Kyun Hota Hai Bina Code Blocks Ke?

Agar spec mein sirf yeh likha ho: "Program user ko greet kare aur time dikhaye" — AI yeh generate kar sakta hai:

- `Hello`
- `Hello World`
- `Hello! Time: 2:30pm`
- `Greetings, human. Current time: 14:30:00`

Code block se exact expected output dikhao — ambiguity khatam!

Fenced Code Block

```text
Task Tracker Menu
1. Add Task
2. View Tasks
3. Exit
Choose an option: _
```

Language Tags — Kyon Zaroori Hain

Tag	Kab Use Karein
`python`	Python code
`bash`	Terminal commands
`text`	Expected program output

`typescript`	TypeScript code
`json`	JSON data
`yaml`	YAML config

> **CRITICAL: Agar Python code ko "text" tag do — AI PEP 8 rules ignore kar deta hai! Language tag = AI ko language rules batata hai.**

Inline Code

Sentence ke andar chhoti cheez ke liye single backtick:

```
Navigate to `cd task-tracker`, then run `python tracker.py`
```

Fenced vs Inline — Kab Kya

Feature	Syntax	Use Kab
Inline Code	`` `code` ``	Variable/file names, short commands in sentences
Fenced Block	` ```` `	Multi-line code, program output, error messages

3 Common Code Block Mistakes

1. **Closing backticks bhool gaye** — sab text code block mein chala jaata hai
2. **Inline code multi-line ke liye** — nahi chalta, fenced use karo
3. **Language tag nahi diya** — AI appropriate rules apply nahi karta

Expert Insight: Specification by Example

> Code blocks act as **executable documentation AND acceptance tests**. Jab AI ka output aapke code block se match kare — implementation correct hai! Concrete examples abstract descriptions se zyada effective hain.

Lesson 5: Links, Images & Emphasis

Links

```
[link text](url)

Example:
Install [Python 3.9](https://python.org) before running.
See [requests docs](https://requests.readthedocs.io) for details.
```

Link Rules:

- Descriptive text likho — destination ka purpose batao
- "click here" ya "link" use mat karo
- URL mein spaces nahi honni chahiye — use `%20`

> **Common Mistake:** URL mein spaces — `[text](my url)` BREAK ho jaata hai. Sahi: `[text](my%20url)`

Images

```
![alt text](image-url)
![App Screenshot](./images/screenshot.png)
```

- `!` prefix image ko link se alag karta hai — zaroori hai!
- Alt text descriptive likhₒ — screen readers ke liye
- Local paths relative use karo: `./images/app.png`

> **REMEMBER: `!` prefix ke bina image render nahi hogi — sirf link banega!**

Text Emphasis

Effect	Syntax	Kab Use Karein
Bold	`**text**` ya `__text__`	Critical requirements, warnings

Italic	`*text*` ya `_text_`	Optional items, lighter emphasis
Bold+Italic	`***text***`	Maximum emphasis

> **Rule:** Bold sirf truly critical items ke liye. Zyada bold use karne se impact khatam ho jaata hai!

Complete Spec Example — Task Tracker App

Yeh ek complete spec hai jo saare 5 lessons use karta hai:

```
# Task Tracker App

## Problem
Users need a simple way to track daily tasks without complicated software.

## Features

### Add Tasks
- Create tasks with title and description
- Set optional due dates
- Assign priority (high, medium, low)

### View Tasks
- Display all tasks with status
- Filter by priority or due date

### Mark Complete
- Mark tasks as done
- Track completion timestamps

### Delete Tasks
- Remove tasks permanently
- **Confirm before deleting** to prevent accidents

## Expected Output
```text
Task Tracker Menu
1. Add Task
2. View Tasks
3. Mark Complete
```
```

```
4. Delete Task
5. Exit
Choose an option: _
```

```
## Installation
1. Install [Python 3.9](https://python.org) or higher
2. Navigate to folder: `cd task-tracker`
3. Run the program: `python tracker.py`
```

> **Is spec mein kya hai:** H1 title, H2 sections, H3 subsections, unordered features, ordered steps, fenced code block, inline code, link, bold emphasis — saab kuch sahi jagah!

Quick Reference — Exam Key Points

Saare Rules Ek Jagah

Rule	Kis Cheez Par
Hamesha `#` ke baad space do	Headings
`#` sirf EK baar use karo (document title)	Headings
Heading levels kabhi skip mat karo	Headings
Unordered ke liye `-`, ordered ke liye `1.`	Lists
`-` ya `1.` ke baad space do	Lists
Ordered sirf tab jab sequence matter kare	Lists
Fenced code blocks hamesha close karo	Code Blocks
Hamesha language tags add karo	Code Blocks
Inline code short references ke liye	Code Blocks
Descriptive link text, "click here" nahi	Links
Images ke liye `!` prefix zaroori hai	Images

Bold sirf truly critical items ke liye	Emphasis

Key Terms

Term	Matlab
Intent Layer	AIDD Layer 1 — aap yahan markdown spec likhte ho
Structured Text	Labels (headings, lists) wala text — humans aur AI dono parse kar sakein
Attention Cues	Markdown headings jo AI ko bataate hain section kahan shuru hota hai
Fenced Code Block	Triple backticks se wrapped code/output
Language Tag	Code block ke baad language identifier (python, bash, text)
CommonMark	Markdown ka base standard (2014)
GFM	GitHub Flavored Markdown — tables, task lists add karta hai
Specification by Example	Code blocks as executable documentation AND acceptance tests
Semantic Anchoring	Code blocks AI ko prevent karte hain hallucinated variations se

Chapter 13 — Markdown Writing Instructions | Panaversity | Exam Preparation Notes | Imran Hassan